Naive Bayes, made easy

A simple way to teach a computer to guess a label, like *spam* or *not spam*, by adding up small clues. By the end you can do it by hand, run it in Python, and know when not to trust it.

11 parts | Beginner → Pro | Example: spam detection

Spam / positive | Ham / negative | Query: the new message

Colours used everywhere: pink = spam, blue = ham (normal mail), yellow star = the new message we want to label.

HOW TO READ THIS

Read the parts in order. Each one builds on the last. Four kinds of boxes help you:

- **KEY IDEA**: the rule to remember.
- **EXAMPLE**: real numbers worked out.
- **WATCH OUT**: a common trap.
- **PRO TIP**: what experts do.

We use **spam detection** all the way through, because it is the classic use. Every number in the hand-worked parts was checked with scikit-learn.

CONTENTS

Beginner: Part 0 Welcome · Part 1 The idea · Part 2 By hand
Intermediate: Part 3 Types of Naive Bayes · Part 4 Real data in Python · Part 5 Tuning alpha
Advanced: Part 6 Seeing why it decides · Part 7 The zero problem · Part 8 Six ways it fails
Pro: Part 9 Pro techniques · Part 10 Revision kit · Appendix

Level 1 · Beginner

Part 0 · Welcome

What is Naive Bayes?

Naive Bayes is a way to put things in groups. You give it a message, and it says "spam" or "ham". It works by looking at each word, asking "does this word look more like spam or more like normal mail?", and adding up the votes.

It is popular because it is **fast**, works with **very little data**, and is easy to explain. Training is just counting words.

Words you will meet

Word	Plain meaning	Spam example
Feature	One clue.	"the word *win* is in the message"
Class / label	The answer we want.	**spam** or **ham**
Prior	Our guess before reading the message.	"40% of all mail is spam"
Likelihood	How often a clue shows up inside a class.	"1 in 3 spam words is *win*"
Posterior	Our updated guess after reading. This is the answer.	"76% spam after seeing *win*"
Vocabulary (V)	The list of different words we know.	6 words in our small example
Bag of words	Count the words, ignore their order.	"win win free" → win: 2, free: 1

A 2-minute probability refresher

- **P(A)** is a number from 0 to 1. A fair die gives P(six) = 1/6.
- **P(A | B)** means "chance of A, *if we already know* B". Among red cards, P(heart | red) = 13/26 = 0.5.
- **Independent** means one event tells you nothing about the other. Two coin flips are independent. "Cloudy" and "rain" are not.
- If A and B are independent, **P(A and B) = P(A) × P(B)**.

Roadmap

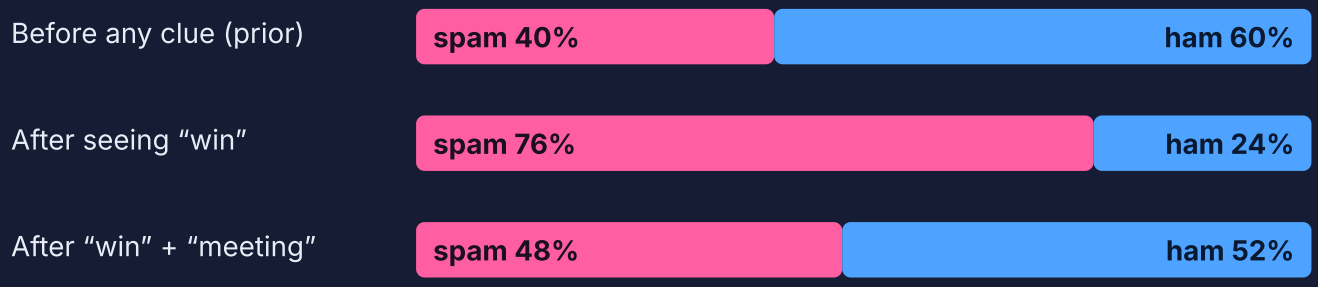
Level	Parts	You will be able to…
Beginner	0 – 2	Explain the idea and label a message with pencil and paper.
Intermediate	3 – 5	Pick the right type, train on real data, tune the one main setting.
Advanced	6 – 8	See why it decides, fix the zero problem, spot six failure cases.
Pro	9 – 10	Use logs and TF-IDF, build it from scratch, pass interview questions.

Level 1 · Beginner

Part 1 · The idea

Think like a detective

A detective starts with a hunch (the **prior**). Then clues arrive one at a time. After each clue the detective changes their mind a little. At the end, the new belief is the **posterior**. Naive Bayes does the same thing, but with numbers.



From our hand example in Part 2. We start at 40% spam. The word "win" pushes it up to 76%. Then "meeting" pulls it back down to 48%, because the two clues disagree.

Bayes' Theorem in words

Bayes' Theorem is the rule for updating a belief. Here it is with our spam names:

$$P(\text{class} \mid \text{words}) = \frac{P(\text{words} \mid \text{class}) \times P(\text{class})}{P(\text{words})}$$

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

- **Prior:** "How common is spam at all?"
- **Likelihood:** "If it really were spam, how likely is it to have these words?"
- **Evidence:** "How common are these words in all mail?" It only makes the final answers add up to 100%.

EXAMPLE · A fever

10% of people have flu. 80% of flu patients have a fever. 12% of people *without* flu also have a fever. You have a fever. How likely is flu?

Fever overall: 0.80×0.10 + 0.12×0.90 = 0.08 + 0.108 = 0.188.

$$P(\text{flu} \mid \text{fever}) = \frac{0.80 \times 0.10}{0.188} \approx 0.43$$

One clue moved the belief from 10% to about 43%. That is all Bayes does: start with a prior, add a clue, get a posterior.

A shortcut: we skip the evidence

To *choose* between spam and ham we only need to know which score is bigger. The evidence is the same for both, so we ignore it:

$$\text{score}(\text{class}) = P(\text{class}) \times P(\text{word}_1 \mid \text{class}) \times P(\text{word}_2 \mid \text{class}) \times \ldots$$

The class with the bigger score wins. (Divide by the total of both scores if you want real percentages.)

Why is it called "naive"?

To be exact, we would need to know how likely the *whole combination* "win + free + prize" is inside spam. With thousands of words there are far too many combinations to count. So Naive Bayes takes a shortcut:

KEY IDEA · Treat every word as a separate clue

Once we know the class, we pretend each word is independent. So we just multiply one small number per word:

$$P(w_1, w_2, \ldots \mid \text{class}) = P(w_1 \mid \text{class}) \times P(w_2 \mid \text{class}) \times \ldots$$

This is "naive" because real words are linked ("free" and "prize" often travel together). But the shortcut still works well, because we only need the right **winner**, not perfect percentages.

WATCH OUT

The assumption is "independent *once we know the class*". It does not say words are unrelated in general.

Level 1 · Beginner

Part 2 · Doing it by hand

No code yet. We train on five tiny emails. The vocabulary has six words: `free, lunch, meeting, prize, project, win`.

Step 1 · The training emails

#	Email	Class
1	win free prize	**spam**
2	win win free	**spam**
3	meeting project	**ham**
4	project lunch meeting	**ham**
5	free lunch meeting	**ham**

Step 2 · Priors

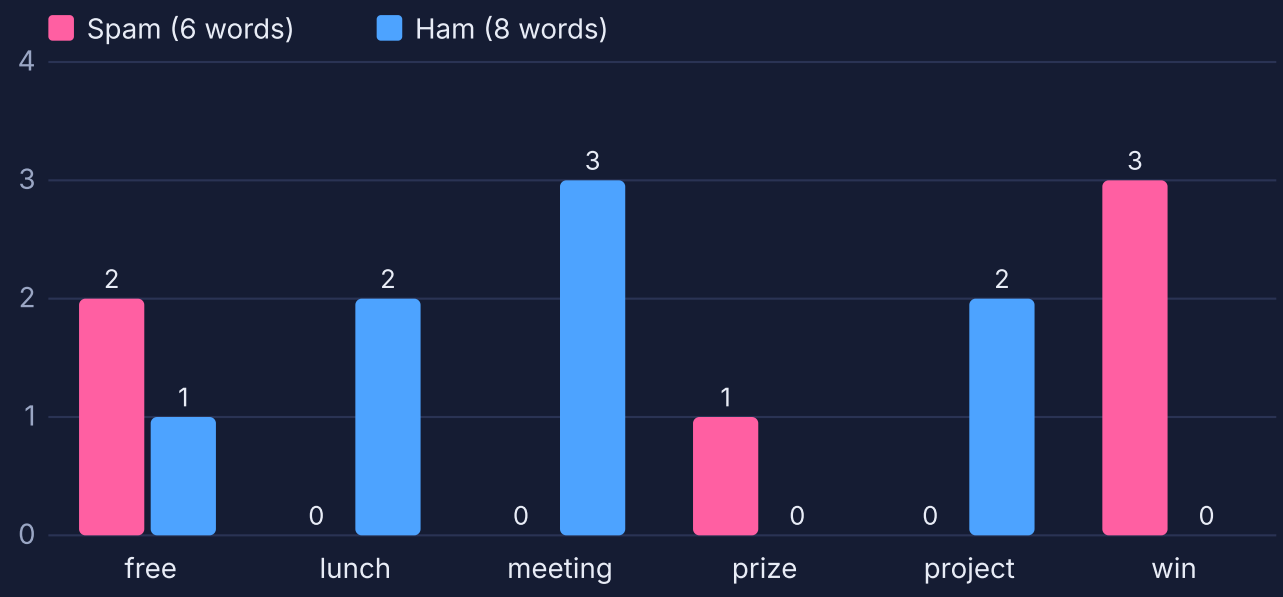
2 of 5 emails are spam and 3 of 5 are ham.

$$P(\text{spam}) = 2/5 = 0.40 \qquad P(\text{ham}) = 3/5 = 0.60$$

Step 3 · Count the words

For each class, count how many times each word appears.

Class	free	lunch	meeting	prize	project	win	Total
Spam	2	0	0	1	0	3	**6**
Ham	1	2	3	0	2	0	**8**



The same table as a picture. Spam uses win, free and prize. Ham uses meeting, project and lunch.

Step 4 · Likelihoods (first try)

Chance of a word inside a class = (times the word appears in that class) ÷ (all words in that class).

$$P(\text{win} \mid \textbf{spam}) = 3/6 = 0.50 \quad P(\text{win} \mid \textbf{ham}) = 0/8 = 0$$
$$P(\text{meeting} \mid \textbf{spam}) = 0/6 = 0 \quad P(\text{meeting} \mid \textbf{ham}) = 3/8 = 0.375$$

Step 5 · Label a new message ★

A new email arrives: ★ **"win meeting"**.

$$\textbf{spam score} = 0.40 \times 0.50 \times 0 = \mathbf{0}$$
$$\textbf{ham score} = 0.60 \times 0 \times 0.375 = \mathbf{0}$$

Both scores are zero, so we cannot decide. This is the **zero problem**: one word that was never seen in a class wipes out everything else, because anything times 0 is 0.

The fix is **Laplace smoothing**: pretend we saw every word one extra time in every class. The amount we add is called **alpha (α)**, and here α = 1.

KEY IDEA · Smoothed likelihood

$$P(\text{word} \mid \text{class}) = \frac{\text{count(word, class)} + \alpha}{\text{words in class} + \alpha \times V}$$

V = 6 is the number of different words. Adding "α × V" below keeps each class's chances adding up to exactly 1.

Step 6 · Try again with smoothing

Chance	Working	Value
P(win | spam)	(3+1) / (6+6)	4/12 = 0.3333
P(meeting | spam)	(0+1) / (6+6)	1/12 = 0.0833
P(win | ham)	(0+1) / (8+6)	1/14 = 0.0714
P(meeting | ham)	(3+1) / (8+6)	4/14 = 0.2857

$$\text{spam score} = 0.40 \times 0.3333 \times 0.0833 = \mathbf{0.01111}$$
$$\text{ham score} = 0.60 \times 0.0714 \times 0.2857 = \mathbf{0.01224}$$

Step 7 · Turn scores into percentages

$$P(\text{spam}) = \frac{0.01111}{0.01111 + 0.01224} \approx \mathbf{0.476} \quad P(\text{ham}) \approx \mathbf{0.524}$$

The model leans a little toward **ham**. That makes sense: "win" says spam, "meeting" says ham, and they nearly cancel. A second message, ★ **"free prize"**, gives about 0.731 spam, a clear spam call.

EXAMPLE · Check with code

This short program gives the same numbers as our hand work:

```
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

docs = ["win free prize", "win win free", "meeting project",
        "project lunch meeting", "free lunch meeting"]
y = np.array([1, 1, 0, 0, 0])            # 1 = spam, 0 = ham

cv = CountVectorizer()
X = cv.fit_transform(docs)
model = MultinomialNB(alpha=1.0).fit(X, y)

print(model.predict_proba(cv.transform(["win meeting"])))
print(model.predict_proba(cv.transform(["free prize"])))
```

```
[[0.52427184 0.47572816]]     # columns: [ham, spam]
[[0.26865672 0.73134328]]
```

The columns follow `model.classes_` : 0 (ham) first, then 1 (spam).

WATCH OUT

A word that was *never in training at all* is simply ignored. Smoothing fixes a different problem: a word seen in one class but not the other. Part 7 explains both.

Level 2 · Intermediate

Part 3 · Types of Naive Bayes

All types work the same way: prior × a likelihood for each feature. They only differ in **how they measure a feature**. Pick the one that matches your data.

1 · Multinomial: word counts

The features are counts (how many times each word appears). This is what we used in Part 2. It is the usual choice for text.

```
from sklearn.naive_bayes import MultinomialNB
MultinomialNB(alpha=1.0)
```

2 · Bernoulli: word is there or not

The features are just yes/no. A special point: it also uses **missing words** as evidence. Not seeing "win" counts a little toward ham. It often suits **short** texts such as SMS or tweets.

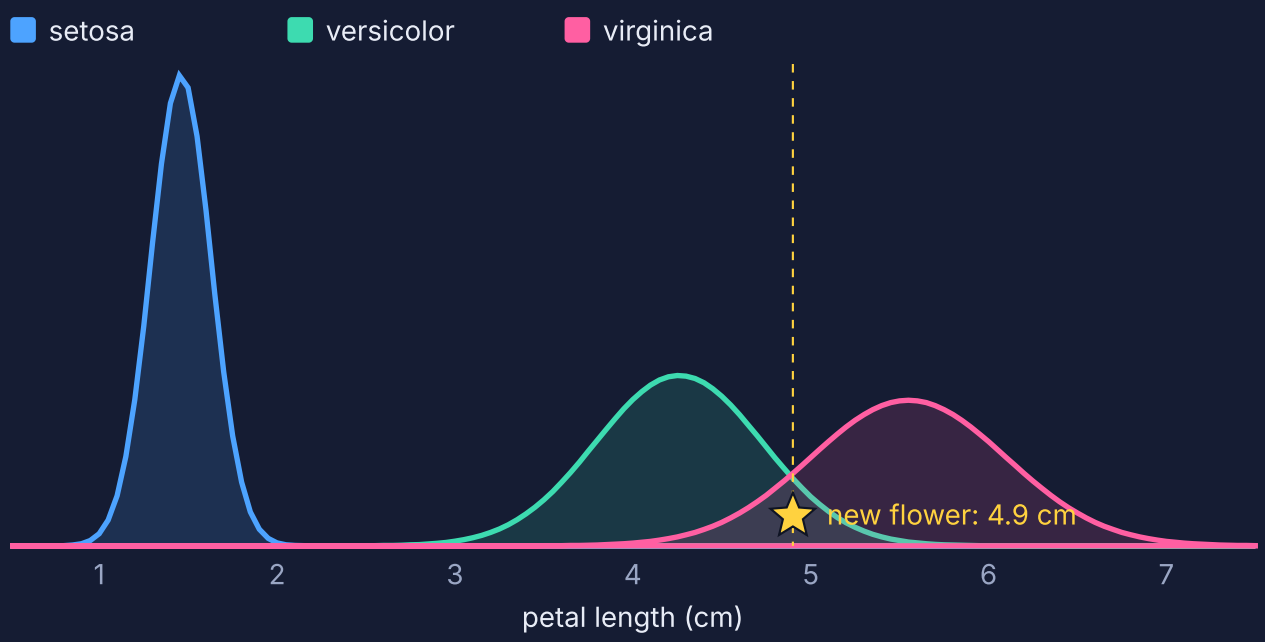
EXAMPLE · Same message, different answer

For "win meeting", Multinomial says 0.476 spam, but Bernoulli says about 0.417. That is because Bernoulli also scores the words that are *missing* (free, lunch, prize, project).

```
from sklearn.naive_bayes import BernoulliNB
BernoulliNB(alpha=1.0, binarize=0.0)    # values above binarize become 1
```

3 · Gaussian: measurements

The features are real numbers (height, petal length, blood pressure). For each class and feature, it learns the **average** and the **spread**, and draws a bell curve. A new value that sits near a class's bell curve scores high for that class.



Petal length for three iris flower types, one bell curve each (teal is only needed because iris has three classes). A new flower of 4.9 cm sits between two curves, so one feature alone almost ties. The other three iris features break the tie. GaussianNB scores about 0.953 on all four features (5-fold cross-validation, which I ran).

```
from sklearn.naive_bayes import GaussianNB
GaussianNB(var_smoothing=1e-9)    # adds a tiny amount to the spread for safety
```

Two extra types

- **ComplementNB:** often better when one class is much smaller than the other.
- **CategoricalNB:** each feature is a category (colour = red / green / blue).

Which one do I pick?

Your features are…	Use
Word counts or TF-IDF	MultinomialNB (default for text)
Word present / absent, short texts	BernoulliNB
Continuous numbers	GaussianNB
Categories	CategoricalNB
Text with unbalanced classes	ComplementNB

PRO TIP

Have numbers and text together? Train one model on each, then multiply their scores. Independent clues multiply, so combining is easy.

Level 2 · Intermediate

Part 4 · Real data in Python

We use the **SMS Spam Collection** (UCI): 5,574 text messages, 747 of them spam (about 13%). Every project follows the same six steps.

> **WATCH OUT · I could not run this part**
>
> My workspace has no internet, so I could not download this dataset. The code uses normal scikit-learn calls, but I give **no output numbers** for it. Expect high accuracy; your exact result depends on the random split.

Step 1 · Load the data

```
import pandas as pd

# Download "SMS Spam Collection" from UCI and unzip it.
# No header: label, a tab, then the message.
df = pd.read_csv("SMSSpamCollection", sep="\t", names=["label", "text"])
df["y"] = (df["label"] == "spam").astype(int)     # spam = 1, ham = 0

print(df.shape)
print(df["y"].value_counts(normalize=True))        # about 0.87 ham / 0.13 spam
```

Step 2 · Split first

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    df["text"], df["y"], test_size=0.2, random_state=42,
    stratify=df["y"])        # keep 13% spam in both halves
```

> **WATCH OUT · Data leakage**
>
> Split *before* building the word list, and build it from the training half only. If test messages leak in, your score looks better than it really is.

Step 3 · Turn text into numbers

A model cannot read words. `CountVectorizer` makes a table: one row per message, one column per word, each cell a count.

```
from sklearn.feature_extraction.text import CountVectorizer

vec = CountVectorizer(lowercase=True, stop_words="english")
Xtr = vec.fit_transform(X_train)    # fit = learn the word list, then count
Xte = vec.transform(X_test)         # only transform: reuse the training word list
```

Step 4 · Train

```
from sklearn.naive_bayes import MultinomialNB
nb = MultinomialNB(alpha=1.0)
nb.fit(Xtr, y_train)                # training = the counting from Part 2
```

Step 5 · Predict

```
pred  = nb.predict(Xte)
proba = nb.predict_proba(Xte)[:, 1]          # chance of spam

new = ["WINNER!! Claim your free prize now", "are we still on for lunch?"]
print(nb.predict(vec.transform(new)))        # expect [1 0]
```

Step 6 · Check the results

```
from sklearn.metrics import classification_report, confusion_matrix
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, target_names=["ham", "spam"]))
```

Term	Meaning here
True positive	Spam correctly caught.
False positive	A good message sent to spam. **The costly mistake.**
False negative	Spam that reached your inbox. Annoying.
Precision	Of the messages we called spam, how many really were spam?
Recall	Of all real spam, how much did we catch?

KEY IDEA · Accuracy can fool you

With 87% ham, a "model" that always says ham is 87% accurate and catches no spam at all. Always look at precision and recall for the rare class.

All in one: a Pipeline

A `Pipeline` joins steps 3 and 4 so the word list is always learned from training data only. Use this in real projects.

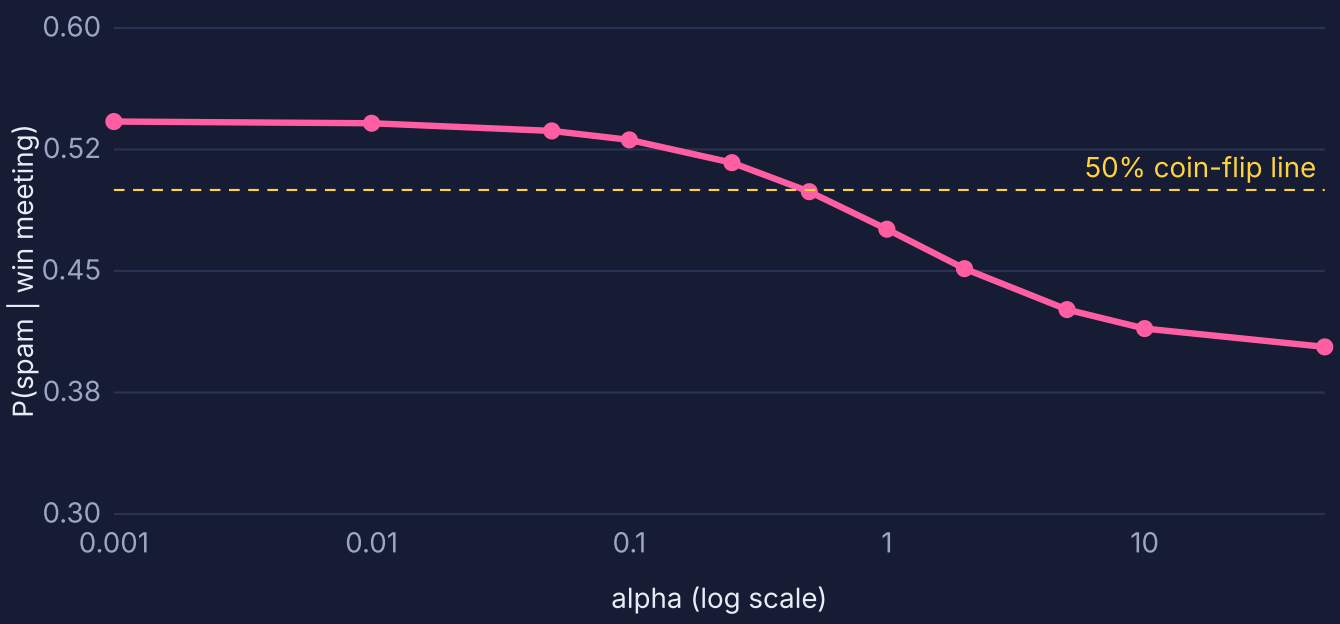
```
from sklearn.pipeline import make_pipeline
clf = make_pipeline(CountVectorizer(stop_words="english"), MultinomialNB())
clf.fit(X_train, y_train)
print(clf.score(X_test, y_test))
```

Level 2 · Intermediate

Part 5 · Tuning alpha

Naive Bayes has very few settings. The main one is **alpha (α)**, the "fake extra count" from Part 2.

Alpha	What happens
Near 0	Almost no smoothing. One unseen word can wipe out a class.
1	Classic add-one. A safe starting point.
Between 0 and 1	Gentle smoothing. Often best when there are many words.
Very large	Every word looks the same in every class, so the model just follows the priors.



Real output from our five-email model: chance of spam for "win meeting" as alpha grows. At tiny alpha it is about 54% spam. Above alpha ≈ 0.5 the answer flips to ham. Very large alpha drifts toward the 40% prior.

Cross-validation in one picture

Never pick alpha by peeking at the test set. Instead, cut the *training* data into 5 parts (folds). Train on 4, check on the 5th, and repeat so each part gets a turn. Average the five scores. The alpha with the best average wins.

Round	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
1	check	train	train	train	train
2	train	check	train	train	train
3	train	train	check	train	train
4	train	train	train	check	train

5	train	train	train	train	check

GridSearchCV with a Pipeline

This tries many settings and picks the best. Because the vectorizer is *inside* the pipeline, each fold builds its own word list, so nothing leaks.

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline

pipe = Pipeline([("vec", CountVectorizer()), ("nb", MultinomialNB())])

grid = {
    "vec__ngram_range": [(1, 1), (1, 2)],      # single words, or words + pairs
    "vec__min_df": [1, 2, 5],                  # drop words in fewer than this many messages
    "nb__alpha": [0.01, 0.1, 0.5, 1.0, 2.0],
}
search = GridSearchCV(pipe, grid, cv=StratifiedKFold(5, shuffle=True, random_state=0),
                      scoring="f1", n_jobs=-1)
search.fit(X_train, y_train)

print(search.best_params_)
print(search.best_score_)
print(search.score(X_test, y_test))     # use the test set ONCE, at the end
```

Why `scoring="f1"`? F1 balances precision and recall for spam, which is right when spam is rare. Use `"precision"` if false alarms are unacceptable.

WATCH OUT · Do not over-trust alpha

On the 1,797-image digits dataset, accuracy moved only from 0.871 to 0.875 while alpha went from 0.0001 to 100. On big datasets the curve is flat. Spend your effort on vectorizer choices (word pairs, min_df, TF-IDF) instead.

PRO TIP

Try alpha on a log scale first (0.001, 0.01, 0.1, 1, 10), then zoom in near the winner. For Gaussian NB, tune `var_smoothing` the same way.

Level 3 · Advanced

Part 6 · Seeing why it decides

A great thing about Naive Bayes is that you can see **which words pushed the decision**.

Each word's vote

Each word adds one number: `log P(word | spam) − log P(word | ham)`. Positive pushes toward **spam**. Negative pushes toward **ham**. (A log just turns multiplying into adding, which is easier to read.)



Word votes from our five-email model. "win" is the strongest spam word (+1.54). "meeting" is the strongest ham word (−1.23). For "win meeting" they nearly cancel. Add the prior (−0.41) and the result leans slightly to ham: the 47.6% / 52.4% from Part 2.

```
import numpy as np
words = np.array(vec.get_feature_names_out())
log_ratio = nb.feature_log_prob_[1] - nb.feature_log_prob_[0]   # spam minus ham
order = np.argsort(log_ratio)
print("Most HAM-like :", words[order[:15]])
print("Most SPAM-like:", words[order[-15:]])
```

How sure is the model?

```
import matplotlib.pyplot as plt
proba = nb.predict_proba(Xte)[:, 1]
plt.style.use("dark_background")
plt.hist(proba[y_test == 0], bins=30, alpha=.8, color="#4da3ff", label="ham")
plt.hist(proba[y_test == 1], bins=30, alpha=.8, color="#ff5fa2", label="spam")
plt.yscale("log"); plt.xlabel("P(spam)"); plt.legend(); plt.show()
```

A good model puts ham near 0 and spam near 1, with few in the middle. Naive Bayes tends to give very extreme numbers, even when it is wrong (see Part 8).

Word clouds

```
# pip install wordcloud
from wordcloud import WordCloud
spam_weights = dict(zip(words, np.exp(nb.feature_log_prob_[1])))
ham_weights  = dict(zip(words, np.exp(nb.feature_log_prob_[0])))

def pink(*a, **k): return "#ff5fa2"
def blue(*a, **k): return "#4da3ff"

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].imshow(WordCloud(background_color="#0e1322",
color_func=pink).generate_from_frequencies(spam_weights))
ax[1].imshow(WordCloud(background_color="#0e1322",
color_func=blue).generate_from_frequencies(ham_weights))
for a in ax: a.axis("off")
plt.show()
```

WATCH OUT · Common is not the same as useful

Words like "the" and "to" are common in both classes, so a plain cloud is full of them. For a more useful picture, size words by `log_ratio` instead.

Can we trust the percentages?

```
from sklearn.calibration import calibration_curve
frac_pos, mean_pred = calibration_curve(y_test, proba, n_bins=10)
plt.plot(mean_pred, frac_pos, "o-", color="#ff5fa2")
plt.plot([0, 1], [0, 1], "--", color="#ffd23f", label="perfect")
plt.xlabel("predicted P(spam)"); plt.ylabel("actual spam fraction"); plt.legend(); plt.show()
```

Points on the dashed line mean "when it says 80%, it is right 80% of the time". Naive Bayes usually bends into an S shape: too bold. Part 8 shows the fix.

Level 3 · Advanced

Part 7 · The zero problem

This is the most common bug when people write Naive Bayes by hand.

What goes wrong

If a word never appeared with a class in training, its chance is 0. Because we multiply, **one zero erases the whole class score**, however strong the other clues are.

EXAMPLE · No smoothing

For ★ "win meeting": spam = 0.40 × 0.50 × **0** = 0, and ham = 0.60 × **0** × 0.375 = 0. Both are gone.

Why Laplace smoothing works

We pretend we saw every word α extra times in every class:

$$P(w \mid c) = \frac{\text{count}(w, c) + \alpha}{N_c + \alpha \times V}$$

Here N_c is the total words in the class. Why the bottom part? If you add up all the tops, you get $N_c + \alpha \times V$, exactly the bottom. So the chances still total 1. Check with the ham row from Part 2 (N = 8, V = 6, α = 1):

$$(1+1)/14 + (2+1)/14 + (3+1)/14 + (0+1)/14 + (2+1)/14 + (0+1)/14 = 14/14 = 1 \checkmark$$

Before and after

Word	spam: before	spam: after	ham: before	ham: after
free	0.333	0.250	0.125	0.143
lunch	**0**	0.083	0.250	0.214
meeting	**0**	0.083	0.375	0.286
prize	0.167	0.167	**0**	0.071
project	**0**	0.083	0.250	0.214
win	0.500	0.333	**0**	0.071

Smoothing takes a little chance from common words and gives it to the "impossible" ones. No zeros are left, and common words barely change. This is also why a big alpha makes all words look alike (Part 5).

Two different "unseen" cases

Seen, but not in this class

"win" never appeared in ham. **Fixed by smoothing.**

Never seen at all

A brand-new word like "cryptozap" is not in the word list, so it is **ignored**. Smoothing is not involved.

PRO TIP

Choose alpha by cross-validation (Part 5). For huge word lists, α near 0.1 often beats 1, because add-one gives too much weight to rare words. For tiny data like ours, α = 1 is fine.

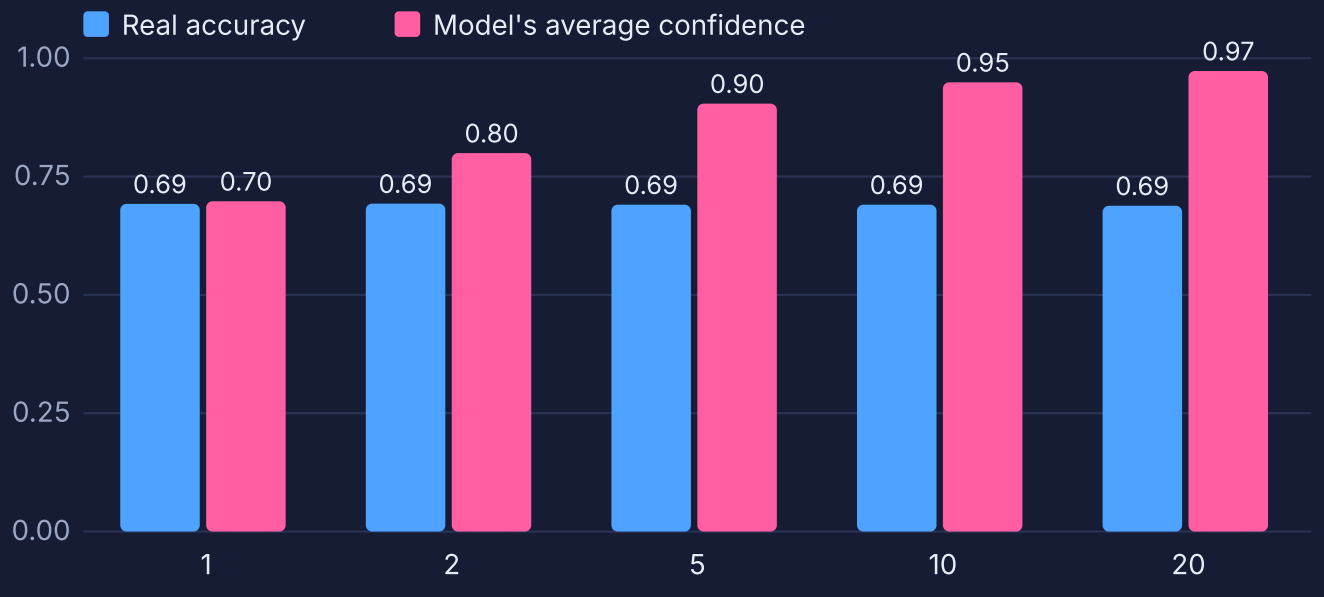
Level 3 · Advanced

Part 8 · Six ways it fails

Knowing when a method fails is as useful as knowing how it works. For each case: **what you see → why → how to fix**.

Failure 1 · Features that repeat each other

What you see: accuracy stays the same, but the model becomes far too sure of itself.
Why: if two features say the same thing, Naive Bayes counts that evidence twice (or ten times).



Real experiment: one weak signal copied k times. Accuracy (blue) stays near 0.69, but confidence (pink) climbs from 0.70 to 0.97. The model gets surer without knowing anything more.

Fixes: remove duplicate features; use feature selection (`SelectKBest` , `chi2`); be careful with word pairs, since "free prize" overlaps with "free" and "prize"; try Logistic Regression, which shares credit between linked features; and calibrate (Failure 5).

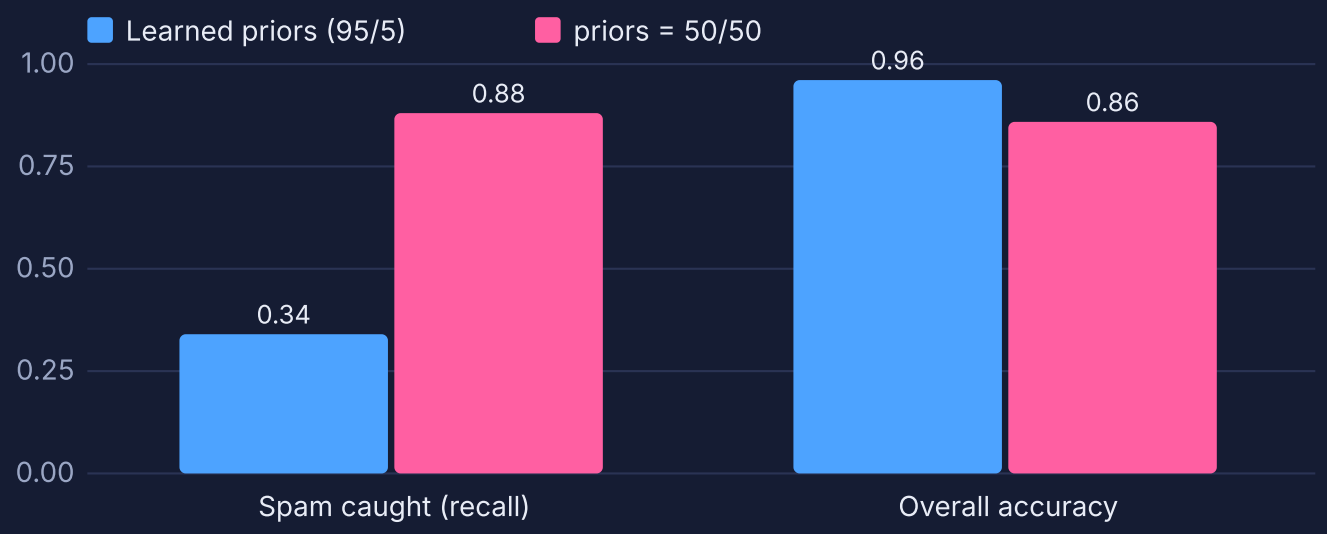
Failure 2 · Numbers that are not bell-shaped

What you see: GaussianNB does badly on skewed features such as income or response time.
Why: it assumes a bell curve, and that fits badly in the tails, where decisions are made.

Fixes: transform the feature (log, `PowerTransformer` , or `QuantileTransformer(output_distribution="normal")`); or cut it into bins with `KBinsDiscretizer` and use `CategoricalNB` . Always plot a histogram per class first.

Failure 3 · Unbalanced classes

What you see: high accuracy, but the rare class is rarely found.
Why: the prior multiplies every score. With 95% ham, ham starts far ahead.



Real experiment: 950 ham, 50 spam. With the learned 95/5 priors (blue), only 34% of spam is caught, yet accuracy is 96%. With 50/50 priors (pink), 88% of spam is caught, but accuracy drops to 86% because more ham is flagged. The right choice depends on which mistake costs more.

Fixes: set `priors=[0.5, 0.5]` (Gaussian) or `fit_prior=False` (Multinomial / Bernoulli); use `ComplementNB`; resample; or move the decision cut-off (for example, call it spam if `proba >= 0.3`). Also make sure your training mix matches real life: if training is 50% spam but real mail is 5%, the learned prior is wrong.

Failure 4 · Tiny data and zero counts

What you see: answers change a lot when you change alpha; rare words decide everything.
Why: counts like 0, 1 and 2 are noisy (Part 7).
Fixes: tune smoothing; use `min_df` to drop very rare words; get more data; remove stop words; use stemming so "winning" and "win" share a count.

Failure 5 · Percentages you cannot trust

What you see: predictions pile up near 0 and 1. "99%" does not mean right 99% of the time.
Why: it comes from Failure 1. Evidence is over-counted, which pushes percentages to the extremes. The *order* of messages is usually still good, which is why Naive Bayes classifies well even when its percentages are poor.

Fix: wrap the model in a calibrator.

```
from sklearn.calibration import CalibratedClassifierCV
calibrated = CalibratedClassifierCV(MultinomialNB(alpha=0.5), method="isotonic", cv=5)
calibrated.fit(Xtr, y_train)    # use method="sigmoid" if you have little data
```

Failure 6 · Word order and combinations

What you see: the model misses patterns made from *combinations* of clues. "Not free" still looks like spam because of "free". "Dog bites man" equals "man bites dog".
Why: bag of words throws away order, and independence stops it from learning combinations. The extreme case is XOR (label is 1 only when two yes/no features differ). I ran Bernoulli NB on XOR data and it scored 0.50, no better than a coin.

Fixes: add word pairs (`ngram_range=(1,2)`) so "not free" becomes one feature; build combination features by hand; or use a model that learns combinations (trees, neural networks).

KEY IDEA · The six at a glance

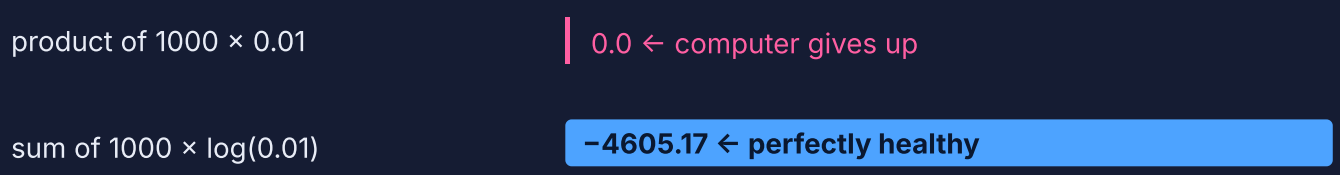
#	Failure	Quick fix
1	Repeated features	Remove duplicates, select features, calibrate
2	Not bell-shaped numbers	Transform or bin
3	Unbalanced classes	Set priors, ComplementNB, move cut-off
4	Tiny data / zero counts	Tune alpha, min_df, more data
5	Over-sure percentages	CalibratedClassifierCV
6	Combinations, word order	Word pairs or a richer model

Level 4 · Pro

Part 9 · Pro techniques

1 · Use logs so numbers do not vanish

A 100-word message multiplies 100 small numbers. A computer can only store numbers down to a point, and then the product becomes exactly 0.0 ("underflow"). Every class then ties at zero.



Real result: multiplying 1000 copies of 0.01 gives 0.0. Adding 1000 copies of log(0.01) gives −4605.17, which is perfectly fine.

KEY IDEA · Work with logs

The class with the biggest score also has the biggest log score, and logs turn multiplying into adding:

$$\log \text{score}(c) = \log P(c) + \Sigma\, x_i \cdot \log P(w_i \mid c)$$

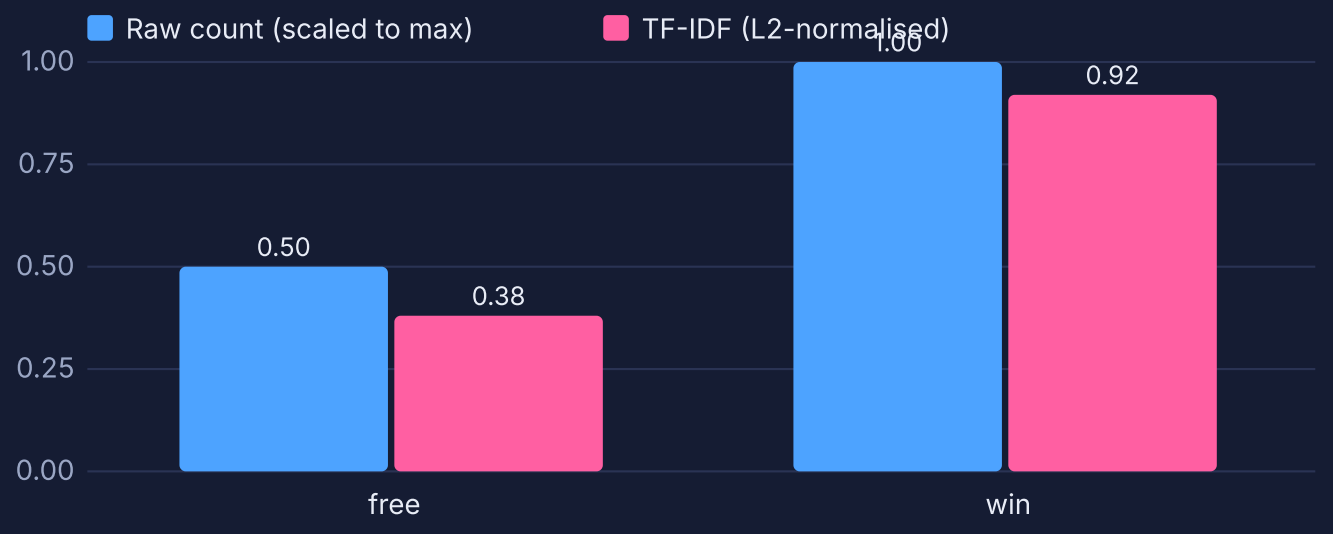
(x_i is how many times word i appears.) To get percentages back safely, subtract the biggest score first, or use `scipy.special.logsumexp`. scikit-learn does this for you. If you write your own, you must do it.

2 · TF-IDF: give rare words more weight

Raw counts treat "the" and "prize" the same. **TF-IDF** makes words that appear in *many* messages count less, and rare, distinctive words count more.

$$\text{tf-idf}(w, d) = \text{tf}(w, d) \times \text{idf}(w) \qquad \text{idf}(w) = \ln \frac{1 + n}{1 + \text{df}(w)} + 1$$

tf = how many times the word is in the message. n = number of messages. df = number of messages that contain the word. This is scikit-learn's default formula, followed by scaling each row to length 1. In our five emails, "prize" is in 1 email: idf = ln(6/2) + 1 = 2.10. "free" is in 3: idf = ln(6/4) + 1 = 1.41.



Message 2, "win win free". Counts (blue) are scaled so the biggest is 1. TF-IDF (pink) pushes the rarer word "win" even further ahead of "free".

```
from sklearn.feature_extraction.text import TfidfVectorizer
pipe = make_pipeline(TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True),
                     MultinomialNB(alpha=0.1))
pipe.fit(X_train, y_train)
```

WATCH OUT

Multinomial NB is built for counts, but TF-IDF usually works as well or better. TF-IDF values are small fractions, so re-tune alpha. The best value is often smaller.

3 · Build it yourself with NumPy

This is the full Multinomial Naive Bayes in about 20 lines. I ran it against scikit-learn on the five emails and got the same answer (0.5243 / 0.4757).

```
import numpy as np

class MyMultinomialNB:
    def __init__(self, alpha=1.0):
        self.alpha = alpha

    def fit(self, X, y):                              # X: (messages, words) counts
        self.classes_ = np.unique(y)
        self.log_prior_ = np.log([(y == c).mean() for c in self.classes_])
        counts = np.array([X[y == c].sum(axis=0) for c in self.classes_]) + self.alpha
        self.log_lik_ = np.log(counts / counts.sum(axis=1, keepdims=True))
        return self

    def _joint_log(self, X):                          # log prior + sum of count * log chance
        return X @ self.log_lik_.T + self.log_prior_

    def predict_proba(self, X):
        j = self._joint_log(X)
        j = j - j.max(axis=1, keepdims=True)          # safety trick
        p = np.exp(j)
        return p / p.sum(axis=1, keepdims=True)

    def predict(self, X):
        return self.classes_[self._joint_log(X).argmax(axis=1)]

mine = MyMultinomialNB(alpha=1).fit(X.toarray(), y)
print(mine.predict_proba(cv.transform(["win meeting"]).toarray()))
```

```
[[0.52427184 0.47572816]]
```

Training is counting. Predicting is one matrix multiplication. That is why Naive Bayes is so fast.

Gaussian Naive Bayes from scratch

It learns the average and spread for each class and feature, then scores with the bell-curve formula. I checked it against `GaussianNB` on iris: the percentages agree to 1e-6 and training accuracy matches at 0.96.

```
from scipy.special import logsumexp

class MyGaussianNB:
    def fit(self, X, y, eps=1e-9):
        self.classes_ = np.unique(y)
        self.log_prior_ = np.log([(y == c).mean() for c in self.classes_])
        self.mu_  = np.array([X[y == c].mean(axis=0) for c in self.classes_])
        self.var_ = np.array([X[y == c].var(axis=0)  for c in self.classes_]) + eps
        return self

    def _joint_log(self, X):
        ll = -0.5 * (np.log(2 * np.pi * self.var_)[None]
                     + (X[:, None, :] - self.mu_[None]) ** 2 / self.var_[None]).sum(axis=2)
        return ll + self.log_prior_

    def predict_proba(self, X):
        j = self._joint_log(X)
        return np.exp(j - logsumexp(j, axis=1, keepdims=True))

    def predict(self, X):
        return self.classes_[self._joint_log(X).argmax(axis=1)]
```

4 · Big data

- **Learn in batches:** `partial_fit` adds counts batch by batch, so data need not fit in memory. Give `classes=[0, 1]` on the first call.
- **HashingVectorizer:** turns words into a fixed number of columns, so no word list is stored. Use `alternate_sign=False` so values stay non-negative.
- **Cost:** training takes time proportional to the total words. Few methods are cheaper.

```
from sklearn.feature_extraction.text import HashingVectorizer
hv = HashingVectorizer(n_features=2**18, alternate_sign=False, norm=None)
nb = MultinomialNB()
for texts, labels in batches:                  # your own generator of (texts, labels)
    nb.partial_fit(hv.transform(texts), labels, classes=[0, 1])
```

5 · Habits for real projects

- **Choose the cut-off, not just the model.** Hiding a real email usually costs more than missing a spam. Pick the cut-off from the precision-recall curve.
- **Save the whole pipeline** (`joblib.dump(pipe, "spam.joblib")`), not just the model, or the word list is lost.
- **Watch for drift.** Spammers change their words. Retraining takes seconds, so do it often.
- **Compare honestly.** Always test against Logistic Regression on the same TF-IDF features.

PRO TIP · Free explanations

If someone asks "why was this message blocked?", add up the word votes from Part 6 and show the top three words. Few models can explain themselves so easily.

Level 4 · Pro

Part 10 · Revision kit

The whole method on one page

THE WHOLE ALGORITHM

1. **Goal:** pick the class with the highest posterior.
2. **Bayes:** posterior ∝ prior × likelihood (skip the evidence).
3. **The naive step:** treat features as independent given the class, so multiply one chance per feature.
4. **Train:** count. Prior = share of each class. Likelihood = (count + α) / (total + α × V).
5. **Predict:** for each class add the log prior and the log likelihoods, then take the biggest.
6. **Types:** Multinomial (counts), Bernoulli (yes/no), Gaussian (numbers), Complement (unbalanced), Categorical.
7. **Main setting:** alpha (or var_smoothing). Tune with cross-validation.
8. **Weak spots:** repeated features, odd-shaped numbers, bad priors, over-sure percentages, no word order.

Interview questions (answers shown)

▼ 1. Why is it called "naive"?

It assumes features are independent once the class is known. That is rarely true in real data.

▼ 2. Why does it work well anyway?

Classifying only needs the right winner, not exact percentages. The errors from the assumption often hurt all classes about equally.

▼ 3. Generative or discriminative?

Generative. It models P(features | class) and P(class), then uses Bayes. Logistic regression is discriminative: it models P(class | features) directly.

▼ 4. What is the zero problem and the fix?

A word never seen with a class gets chance 0, which zeroes the whole product. Fix it with smoothing: add α to counts and α × V to the total.

▼ 5. Why use logs?

Multiplying many small numbers underflows to 0. Logs turn products into sums, which are stable, and the winner stays the same.

▼ 6. Multinomial vs Bernoulli?

Multinomial uses counts and ignores missing words. Bernoulli uses yes/no and counts missing words as evidence. Bernoulli often suits short texts.

▼ 7. When use Gaussian NB, and what goes wrong?

For continuous features that are roughly bell-shaped within each class. It struggles with skewed or multi-peak features; transform or bin them.

▼ 8. What does a large alpha do?

It flattens the likelihoods, so words lose their power and predictions drift to the priors (underfitting). Too small an alpha overfits to rare counts.

▼ 9. Why are the percentages poorly calibrated?

Linked features are counted as separate evidence, which pushes percentages toward 0 or 1. Use `CalibratedClassifierCV`.

▼ 10. How does class imbalance hurt, and what helps?

The prior favours the big class. Change priors, use ComplementNB, resample, or move the cut-off. Judge by precision and recall, not accuracy.

▼ 11. How many numbers does it learn?

About C × V likelihoods plus C priors (C classes, V features). A full joint model would need exponentially many.

▼ 12. Naive Bayes vs logistic regression for text?

Naive Bayes trains in one pass, handles tiny data, and has no learning rate. Logistic regression copes better with linked features and gives better percentages, and often wins with plenty of data.

▼ 13. Can it handle streaming data?

Yes. Counts just add up, so `partial_fit` updates the model batch by batch.

▼ 14. What about a test word not seen in training?

It is ignored. This is different from the zero problem, which smoothing solves.

▼ 15. Does feature scaling matter?

Not for Gaussian NB, since each feature gets its own average and spread. For Multinomial NB, values must not be negative.

Practice problems

▼ Problem A · Compute by hand

Use the five emails from Part 2 with α = 1. Label ★ "free lunch". Give both scores.

Answer: P(free | spam) = 3/12 and P(lunch | spam) = 1/12, so spam = 0.40 × 0.25 × 0.0833 = 0.00833. P(free | ham) = 2/14 and P(lunch | ham) = 3/14, so ham = 0.60 × 0.1429 × 0.2143 = 0.01837. As percentages: ham ≈ 0.688, spam ≈ 0.312. Predict ham.

▼ Problem B · Explain the flip

In the alpha chart, why does P(spam | "win meeting") fall as alpha goes from 0.5 to 50?

Answer: Bigger alpha flattens every word's chance toward 1/V. The strong spam signal of "win" gets watered down, and the model drifts toward the 40% prior.

▼ Problem C · Diagnose

A fraud model is 99% accurate (1% of cases are fraud) but flags almost no fraud. Name two likely causes and fixes.

Answer: The 99/1 prior, and accuracy being a misleading score. Measure recall and precision, set equal priors or move the cut-off, and try ComplementNB or resampling.

▼ Problem D · Code it

Add a `fit_prior=False` option to `MyMultinomialNB` that uses equal priors. **Hint:** replace `log_prior_` with `np.log(np.full(len(classes_), 1/len(classes_)))`.

Should I use Naive Bayes?

USE IT WHEN…

☐ You want a quick baseline before anything complicated.

☐ Your data is text with thousands of words but not many rows.

☐ Training data is small or arrives as a stream.

☐ Speed and low memory matter, or you retrain often.

☐ You want to explain results word by word.

☐ The order of results matters more than exact percentages.

THINK TWICE WHEN…

☐ Features are strongly linked and you need trustworthy percentages.

☐ The signal is in combinations or word order (sarcasm, "not good").

☐ Numbers are very skewed and you have no time to transform them.

☐ You have lots of data and want the last bit of accuracy: compare with logistic regression, gradient boosting or a transformer.

Extra

Appendix · Cheat sheets

Formulas

Idea	Formula
Bayes' rule	$P(c \mid x) = P(x \mid c)\,P(c) / P(x)$
Naive Bayes choice	$\hat{c} = \text{argmax}_c\, P(c) \prod P(x_i \mid c)$
Log form	$\hat{c} = \text{argmax}_c\, [\log P(c) + \Sigma \log P(x_i \mid c)]$
Multinomial likelihood	$(\text{count}(w,c) + \alpha) / (N_c + \alpha V)$
Bernoulli likelihood	(messages in class with w + α) / (messages in class + 2α)
Gaussian likelihood	bell curve with the class's average and spread
TF-IDF (sklearn)	tf × (ln((1+n)/(1+df)) + 1), then scale each row to length 1

scikit-learn settings

Class	Key settings
MultinomialNB	`alpha`, `fit_prior`, `class_prior`
BernoulliNB	`alpha`, `binarize`, `fit_prior`, `class_prior`
GaussianNB	`priors`, `var_smoothing`
ComplementNB	`alpha`, `norm`, `fit_prior`
CategoricalNB	`alpha`, `min_categories`
CountVectorizer / TfidfVectorizer	`ngram_range`, `min_df`, `max_df`, `stop_words`, `sublinear_tf` (TF-IDF)

Common mistakes

- Building the word list on all data before splitting (leakage).
- Reading `predict_proba` columns in the wrong order. Check `model.classes_`.
- Giving negative values to MultinomialNB (for example after standard scaling).
- Trusting accuracy when classes are unbalanced.
- Treating Naive Bayes percentages as reliable.
- Choosing alpha using the test set.

What was checked: every number in Parts 1–3, 5, 7, 8 and 9 comes from running code in a sandbox with scikit-learn. Part 4's SMS code could not be run (no internet), so it has no output numbers.